¡Llegamos a la Tanda 3: El Cuerpo y la Validación Semántica!

Al terminar esta tanda, tu compilador podrá leer los archivos .ins completos de las criaturas de ejemplo (uruk.ins, hobbit.ins, ent.ins, dwarf.ins), y detectará de inmediato cualquier error de sintaxis o semántica (etiquetas duplicadas, saltos a etiquetas inexistentes, asignaciones prohibidas a percepciones, etc.).

---

1. Definir los Catálogos Base (constants.py)

Para cumplir con la directiva del PDF (*"añadir una acción o percepción nueva debería ser registrarla sin tocar la lógica existente"*), vamos a centralizar los catálogos del lenguaje en un archivo limpio.

Crea el archivo src/language/constants.py:

Python

```
"""Catálogos base del lenguaje Instinct (percepciones, constantes, acciones y funciones)."""


# Constantes del mundo (valores enteros según el PDF)
WORLD_CONSTANTS = {
    "NONE": -1,
    "GROUND": 0,
    "OBJECT": 1,
    "ALLY": 2,
    "ENEMY": 3,
}


# Percepciones de solo lectura (no se les puede asignar un valor)
READONLY_PERCEPTIONS = {
    # Propias
    "health", "age", "lifespan", "vision", "species", "faction",
    "x", "y", "terrain_here", "reserve_here",
    # Del mundo
    "width", "height", "tick",
    "total_creatures", "total_allies", "total_enemies",
    "random",
```

```
    # Del entorno cercano
    "allies_near", "enemies_near", "objects_near",
    "enemy_dist", "enemy_dx", "enemy_dy",
    "ally_dist", "ally_dx", "ally_dy",
    "object_dist", "object_dx", "object_dy",
    "roars_near", "last_roar",
}

# Acciones base obligatorias y su cantidad exacta de argumentos
BUILTIN_ACTIONS = {
    "wait": 1,        # wait(n)
    "move": 3,        # move(dx, dy, speed)
    "attack": 3,      # attack(dx, dy, damage)
    "consume": 4,     # consume(dx, dy, damage, heal)
    "reproduce": 3,   # reproduce(dx, dy, health)
    "roar": 1,        # roar(value)
    "say": 1,         # say(value)
}

# Funciones reservadas en expresiones y su cantidad exacta de argumentos
BUILTIN_FUNCTIONS = {
    "see": 2,         # see(x, y)
    "name": 2,        # name(x, y)
}
```

---

2. El Parser Completo (src/language/parser.py)

Ahora actualizamos src/language/parser.py para parsear las 5 formas de línea del cuerpo:

1. **Etiqueta: name:**
2. **Salto incondicional: goto name**
3. **Salto condicional: if expr goto name**

4. **Asignación: name = expr**
5. **Acción: action(arg1, arg2, ...)**

Y aplicamos las validaciones en tiempo de compilación que exige el PDF.

Reemplaza todo el contenido de src/language/parser.py con esto:

Python

```
from typing import List, Dict, Set
from src.language.tokens import Token, TokenType
from src.language.ast_nodes import (
    CompiledProgram,
    Instruction,
    LabelInst,
    GotoInst,
    ConditionalGotoInst,
    AssignmentInst,
    ActionInst,
    Expression,
    LiteralInt,
    LiteralString,
    VariableRef,
    UnaryOp,
    BinaryOp,
    FunctionCall,
)
from src.language.errors import CompileError
from src.language.constants import (
    READONLY_PERCEPTIONS,
    WORLD_CONSTANTS,
    BUILTIN_ACTIONS,
    BUILTIN_FUNCTIONS,
)
```

```
REQUIRED_HEADER_KEYS = {"creature", "faction", "health", "vision", "lifespan"}



class Parser:
    """
    Parser por descenso recursivo para el lenguaje Instinct.
    Convierte una lista de tokens en un CompiledProgram validado.
    """

    def __init__(self, tokens: List[Token]):
        self.tokens = tokens
        self.current = 0
        self.program = CompiledProgram()

    # ========================================================================
    # PUNTO DE ENTRADA PRINCIPAL
    # ========================================================================

    def parse(self) -> CompiledProgram:
        """Compila cabecera y cuerpo, valida semántica y devuelve el programa listo."""
        self._skip_newlines()
        self._parse_header()
        self._parse_body()
        self._validate_semantics()
        return self.program

    # ========================================================================
    # 1. CABECERA
```

```
# ============================================================================

def _parse_header(self) -> None:
    seen_keys: Set[str] = set()

    if not self._check_identifier("creature"):
        raise CompileError(
            "La primera línea del archivo debe ser 'creature NombreEspecie'",
            self._peek().line
        )

    while not self._is_at_end():
        if self._is_start_label():
            break

        key_token = self._advance()

        if key_token.type != TokenType.IDENTIFIER:
            raise CompileError(
                f"Se esperaba una clave de cabecera (creature, faction, health, vision, lifespan), "
                f"pero se encontró: '{key_token.value}'",
                key_token.line
            )

        key = key_token.value

        if key not in REQUIRED_HEADER_KEYS:
            raise CompileError(
                f"Clave de cabecera desconocida: '{key}'. "
                f"Se esperaba una de: {REQUIRED_HEADER_KEYS}",
```

```
                key_token.line
            )

        if key in seen_keys:
            raise CompileError(
                f"La clave de cabecera '{key}' está declarada más de una vez",
                key_token.line
            )

        seen_keys.add(key)
        self._parse_header_value(key, key_token.line)
        self._consume_newline(f"Se esperaba salto de línea tras la declaración de '{key}'")
        self._skip_newlines()

    missing = REQUIRED_HEADER_KEYS - seen_keys
    if missing:
        raise CompileError(
            f"Faltan claves obligatorias en la cabecera: {missing}",
            self._peek().line
        )

def _parse_header_value(self, key: str, line: int) -> None:
    value_token = self._advance()

    if key in ("creature", "faction"):
        if value_token.type != TokenType.IDENTIFIER:
            raise CompileError(
                f"'{key}' requiere un nombre (identificador), no '{value_token.value}'",
                line
            )
```

```
        if key == "creature":
            self.program.creature_name = value_token.value
        else:
            self.program.faction = value_token.value

    elif key in ("health", "vision", "lifespan"):
        if value_token.type != TokenType.NUMBER:
            raise CompileError(
                f"'{key}' requiere un número entero, no '{value_token.value}'",
                line
            )
        num = value_token.value

        if key == "health" and num <= 0:
            raise CompileError("'health' debe ser mayor que 0", line)
        if key == "vision" and num < 1:
            raise CompileError("'vision' debe ser mayor o igual que 1", line)
        if key == "lifespan" and num <= 0:
            raise CompileError("'lifespan' debe ser mayor que 0", line)

        if key == "health":
            self.program.health = num
        elif key == "vision":
            self.program.vision = num
        else:
            self.program.lifespan = num

# ============================================================================
# 2. CUERPO DE INSTRUCCIONES
# ============================================================================
```

```
    def _parse_body(self) -> None:
        """Parsea todas las líneas a partir de start: hasta el EOF."""
        if not self._is_start_label():
            raise CompileError("El cuerpo debe comenzar obligatoriamente con la etiqueta 'start:'", self._peek().line)

        while not self._is_at_end():
            self._skip_newlines()
            if self._is_at_end():
                break

            inst = self._parse_instruction_line()
            if inst is not None:
                self.program.instructions.append(inst)

            # Consumir el salto de línea al final de la instrucción
            if not self._is_at_end():
                self._consume_newline("Se esperaba salto de línea tras la instrucción")

    def _parse_instruction_line(self) -> Optional[Instruction]:
        token = self._peek()
        line = token.line

        # 1. Salto condicional: if <expr> goto <label>
        if self._match(TokenType.IF):
            condition = self.parse_expression()
            self._consume(TokenType.GOTO, "Se esperaba 'goto' después de la condición en 'if'")
            label_token = self._consume(TokenType.IDENTIFIER, "Se esperaba el nombre de la etiqueta tras 'goto'")
            return ConditionalGotoInst(condition, label_token.value, line)
```

```
        # 2. Salto incondicional: goto <label>
        if self._match(TokenType.GOTO):
            label_token = self._consume(TokenType.IDENTIFIER, "Se esperaba el nombre de la etiqueta tras 'goto'")
            return GotoInst(label_token.value, line)

        # Si empieza con IDENTIFIER, puede ser Etiqueta, Asignación o Acción
        if self._check(TokenType.IDENTIFIER):
            name_token = self._advance()
            name = name_token.value

            # Caso 3: Etiqueta (name:)
            if self._match(TokenType.COLON):
                if name in self.program.labels:
                    raise CompileError(f"Etiqueta duplicada: '{name}'", line)
                # La etiqueta apunta al índice de la instrucción actual en la lista
                self.program.labels[name] = len(self.program.instructions)
                return LabelInst(name, line)

            # Caso 4: Asignación (name = expr)
            if self._match(TokenType.ASSIGN):
                if name in READONLY_PERCEPTIONS:
                    raise CompileError(f"No se puede asignar un valor a la percepción de solo lectura '{name}'", line)
                if name in WORLD_CONSTANTS:
                    raise CompileError(f"No se puede asignar un valor a la constante '{name}'", line)

                value_expr = self.parse_expression()
                return AssignmentInst(name, value_expr, line)
```

```
            # Caso 5: Acción (name(arg1, arg2, ...))
            if self._match(TokenType.LPAREN):
                if name not in BUILTIN_ACTIONS:
                    raise CompileError(f"Acción desconocida: '{name}'", line)

                args = self._parse_argument_list()
                self._consume(TokenType.RPAREN, f"Se esperaba ')' tras los argumentos de '{name}'")

                expected_args = BUILTIN_ACTIONS[name]
                if len(args) != expected_args:
                    raise CompileError(
                        f"La acción '{name}' espera {expected_args} argumento(s), pero se le pasaron {len(args)}",
                        line
                    )

                return ActionInst(name, args, line)

            raise CompileError(
                f"Línea no válida o desconocida: '{token.value if token.value is not None else token.type.name}'",
                line
            )

    # ============================================================================
    # 3. EXPRESIONES
    # ============================================================================

    def parse_expression(self) -> Expression:
        return self._parse_or()
```

```
def _parse_or(self) -> Expression:
    expr = self._parse_and()
    while self._match(TokenType.OR):
        right = self._parse_and()
        expr = BinaryOp("or", expr, right)
    return expr

def _parse_and(self) -> Expression:
    expr = self._parse_comparison()
    while self._match(TokenType.AND):
        right = self._parse_comparison()
        expr = BinaryOp("and", expr, right)
    return expr

def _parse_comparison(self) -> Expression:
    expr = self._parse_additive()
    while self._peek().type in (
        TokenType.LESS,
        TokenType.LESS_EQUAL,
        TokenType.GREATER,
        TokenType.GREATER_EQUAL,
        TokenType.EQUAL_EQUAL,
        TokenType.BANG_EQUAL,
    ):
        op_token = self._advance()
        op_symbols = {
            TokenType.LESS: "<",
            TokenType.LESS_EQUAL: "<=",
            TokenType.GREATER: ">",
            TokenType.GREATER_EQUAL: ">=",
```

```
            TokenType.EQUAL_EQUAL: "==",
            TokenType.BANG_EQUAL: "!=",
        }
        right = self._parse_additive()
        expr = BinaryOp(op_symbols[op_token.type], expr, right)
    return expr

def _parse_additive(self) -> Expression:
    expr = self._parse_multiplicative()
    while self._peek().type in (TokenType.PLUS, TokenType.MINUS):
        op_token = self._advance()
        op = "+" if op_token.type == TokenType.PLUS else "-"
        right = self._parse_multiplicative()
        expr = BinaryOp(op, expr, right)
    return expr

def _parse_multiplicative(self) -> Expression:
    expr = self._parse_unary()
    while self._peek().type in (TokenType.STAR, TokenType.SLASH, TokenType.PERCENT):
        op_token = self._advance()
        op_map = {
            TokenType.STAR: "*",
            TokenType.SLASH: "/",
            TokenType.PERCENT: "%",
        }
        right = self._parse_unary()
        expr = BinaryOp(op_map[op_token.type], expr, right)
    return expr

def _parse_unary(self) -> Expression:
```

```
        if self._peek().type in (TokenType.MINUS, TokenType.NOT):
            op_token = self._advance()
            op = "-" if op_token.type == TokenType.MINUS else "not"
            operand = self._parse_unary()
            return UnaryOp(op, operand)
        return self._parse_primary()

    def _parse_primary(self) -> Expression:
        token = self._peek()

        if self._match(TokenType.NUMBER):
            return LiteralInt(token.value)

        if self._match(TokenType.STRING):
            return LiteralString(token.value)

        if self._match(TokenType.IDENTIFIER):
            name = token.value

            if self._match(TokenType.LPAREN):
                if name not in BUILTIN_FUNCTIONS:
                    raise CompileError(f"Función desconocida en expresión: '{name}'", token.line)

                args = self._parse_argument_list()
                self._consume(TokenType.RPAREN, f"Se esperaba ')' tras los argumentos de '{name}'")

                expected_args = BUILTIN_FUNCTIONS[name]
                if len(args) != expected_args:
                    raise CompileError(
                        f"La función '{name}' espera {expected_args} argumento(s), pero recibió
{len(args)}",
```

```
                token.line
            )
        return FunctionCall(name, args)

    return VariableRef(name)

if self._match(TokenType.LPAREN):
    expr = self.parse_expression()
    self._consume(TokenType.RPAREN, "Se esperaba ')' para cerrar la expresión")
    return expr

raise CompileError(
    f"Expresión no válida o inesperada: '{token.value if token.value is not None else token.type.name}'",
    token.line
)

def _parse_argument_list(self) -> List[Expression]:
    args: List[Expression] = []
    if self._check(TokenType.RPAREN):
        return args

    args.append(self.parse_expression())
    while self._match(TokenType.COMMA):
        args.append(self.parse_expression())
    return args

# ==============================================================================
# 4. VALIDACIÓN SEMÁNTICA POST-PARSING
# ==============================================================================
```

```
    def _validate_semantics(self) -> None:
        """Verifica coherencia global del archivo compilado."""
        # 1. Comprobar que existe la etiqueta 'start'
        if "start" not in self.program.labels:
            raise CompileError("No se encontró la etiqueta obligatoria 'start:' en el cuerpo", 1)

        # 2. Comprobar que todos los 'goto' apunten a etiquetas existentes
        for inst in self.program.instructions:
            if isinstance(inst, GotoInst):
                if inst.target not in self.program.labels:
                    raise CompileError(f"Salto 'goto {inst.target}' a una etiqueta inexistente",
inst.line_number)
            elif isinstance(inst, ConditionalGotoInst):
                if inst.target not in self.program.labels:
                    raise CompileError(f"Salto condicional 'goto {inst.target}' a una etiqueta inexistente",
inst.line_number)

    # ========================================================================
    # UTILIDADES DE NAVEGACIÓN
    # ========================================================================

    def _peek(self, offset: int = 0) -> Token:
        pos = self.current + offset
        if pos >= len(self.tokens):
            return self.tokens[-1]
        return self.tokens[pos]

    def _advance(self) -> Token:
        token = self.tokens[self.current]
        if not self._is_at_end():
            self.current += 1
```

```
    return token

def _match(self, token_type: TokenType) -> bool:
    if self._check(token_type):
        self._advance()
        return True
    return False

def _consume(self, token_type: TokenType, error_msg: str) -> Token:
    if self._check(token_type):
        return self._advance()
    raise CompileError(error_msg, self._peek().line)

def _is_at_end(self) -> bool:
    return self._peek().type == TokenType.EOF

def _check(self, token_type: TokenType) -> bool:
    return self._peek().type == token_type

def _check_identifier(self, name: str) -> bool:
    return self._check(TokenType.IDENTIFIER) and self._peek().value == name

def _is_start_label(self) -> bool:
    return (
        self._check_identifier("start")
        and self._peek(1).type == TokenType.COLON
    )

def _skip_newlines(self) -> None:
    while self._check(TokenType.NEWLINE):
```

```
            self._advance()

    def _consume_newline(self, error_msg: str) -> None:
        if not self._check(TokenType.NEWLINE) and not self._is_at_end():
            raise CompileError(error_msg, self._peek().line)
        if self._check(TokenType.NEWLINE):
            self._advance()
```

---

3. La Gran Prueba: Compilar los 4 Bichos del PDF

Vamos a crear un test completo que compile el Uruk, el Hobbit, el Ent y el Enano (Dwarf), y además pruebe que el compilador capture todos los errores semánticos posibles.

Actualiza tests/test_parser.py:

Python

```
from src.language.lexer import Lexer
from src.language.parser import Parser
from src.language.errors import CompileError


# ============================================================================
# CÓDIGO DE LAS CRIATURAS DEL PDF
# ============================================================================

URUK_CODE = """# uruk.ins
creature Uruk
faction isengard
health 80
vision 6
lifespan 400

start:
if health < 20 goto flee
```

```
if enemy_dist == 1 goto bite
if enemy_dist > 0 goto hunt
if health > 70 and allies_near < 2 goto breed
if allies_near > 3 goto celebrate

wander:
move(random % 3 - 1, random % 3 - 1, 1)
goto start

bite:
consume(enemy_dx, enemy_dy, 15, 15)
goto start

hunt:
move(enemy_dx, enemy_dy, 2)
goto start

flee:
move(-enemy_dx, -enemy_dy, 3)
goto start

breed:
if see(x, y + 1) != GROUND goto wander
reproduce(0, 1, 30)
goto start

celebrate:
say("meat is back on the menu")
roar("we are the fighting Uruk-hai")
goto start
```

"""

HOBBIT_CODE = """# hobbit.ins

creature Hobbit

faction shire

health 60

vision 5

lifespan 300

start:

if enemy_dist >= 0 goto flee

if health > 45 and total_allies < 20 goto breed

if reserve_here > 0 and health < 80 goto second_breakfast

if ally_dist > 1 goto go_ally

wait(1)

goto start

flee:

say("they are taking us to Isengard")

move(-enemy_dx, -enemy_dy, 2)

goto start

second_breakfast:

consume(0, 0, 10, 10)

goto start

go_ally:

move(ally_dx, ally_dy, 1)

goto start

```
breed:
if see(x + 1, y) == GROUND goto breed_right
if see(x - 1, y) == GROUND goto breed_left
wait(1)
goto start

breed_right:
reproduce(1, 0, 20)
goto start

breed_left:
reproduce(-1, 0, 20)
goto start
"""

ENT_CODE = """# ent.ins
creature Ent
faction fangorn
health 150
vision 4
lifespan 1000

start:
home_x = x
home_y = y

watch:
if enemy_dist == 1 goto hit
if enemies_near > 0 and roars_near == 0 goto warn
if x != home_x or y != home_y goto go_home
```

```
if health < 150 goto roots
wait(1)
goto watch

hit:
attack(enemy_dx, enemy_dy, 12)
goto watch

warn:
roar("hoom, hom, do not be hasty")
goto watch

roots:
consume(0, 0, 3, 3)
goto watch

go_home:
dx = (home_x > x) - (home_x < x)
dy = (home_y > y) - (home_y < y)
move(dx, dy, 1)
goto watch
"""

DWARF_CODE = """# dwarf.ins
creature Dwarf
faction erebor
health 100
vision 3
lifespan 600
```

```
start:
if last_roar == "khazad ai-menu" goto rescue
if object_dist == 1 and health < 100 goto mine
dx = random % 3 - 1
dy = random % 3 - 1
steps = 0

walk:
if steps >= 5 goto start
if enemy_dist == 1 goto shout
move(dx, dy, 1)
steps = steps + 1
goto walk

mine:
if name(x + object_dx, y + object_dy) != "Rock" goto start
consume(object_dx, object_dy, 20, 20)
goto start

shout:
roar("khazad ai-menu")
goto start

rescue:
say("and my axe")
move(ally_dx, ally_dy, 2)
goto start
"""
```

```
def compile_creature(code: str, name: str):
    print(f"--> Compilando {name}...")
    tokens = Lexer(code).tokenize()
    program = Parser(tokens).parse()
    print(f"    ✓ {name} compilado con éxito: {len(program.instructions)} instrucciones, etiquetas: {list(program.labels.keys())}")
    return program


def test_compile_all_examples():
    print("========================================")
    print("   COMPILANDO CRIATURAS DE EJEMPLO      ")
    print("========================================")
    compile_creature(URUK_CODE, "Uruk")
    compile_creature(HOBBIT_CODE, "Hobbit")
    compile_creature(ENT_CODE, "Ent")
    compile_creature(DWARF_CODE, "Dwarf")
    print("\n✅ ¡Todos los ejemplos del profesor compilan perfectamente!")


def test_compile_errors():
    print("\n========================================")
    print("   PROBANDO ERRORES DE COMPILACIÓN      ")
    print("========================================")

    # 1. Asignar a percepción
    try:
        code = "creature X\nfaction F\nhealth 10\nvision 1\nlifespan 10\nstart:\nhealth = 50"
        Parser(Lexer(code).tokenize()).parse()
        print("❌ Error: debió fallar asignación a health")
```

```
    except CompileError as e:
        print(f"✓ Error capturado: {e}")

    # 2. Goto a etiqueta que no existe
    try:
        code = "creature X\nfaction F\nhealth 10\nvision 1\nlifespan 10\nstart:\ngoto fantasma"
        Parser(Lexer(code).tokenize()).parse()
        print("❌ Error: debió fallar goto inexistente")
    except CompileError as e:
        print(f"✓ Error capturado: {e}")

    # 3. Acción con número incorrecto de argumentos
    try:
        code = "creature X\nfaction F\nhealth 10\nvision 1\nlifespan 10\nstart:\nmove(1, 2)"
        Parser(Lexer(code).tokenize()).parse()
        print("❌ Error: debió fallar move con 2 argumentos")
    except CompileError as e:
        print(f"✓ Error capturado: {e}")

    # 4. Etiqueta duplicada
    try:
        code = "creature X\nfaction F\nhealth 10\nvision 1\nlifespan
10\nstart:\nwait(1)\nstart:\nwait(1)"
        Parser(Lexer(code).tokenize()).parse()
        print("❌ Error: debió fallar etiqueta duplicada")
    except CompileError as e:
        print(f"✓ Error capturado: {e}")


if __name__ == "__main__":
```

test_compile_all_examples()

test_compile_errors()